

新增一個文字欄位 Note

DDD 訂位系統 — 上線後的加欄位手冊(手動修改版,不重新產生 solution)

創業惟艱,守業更難。
從零開始建一個專案,錯了可以砍掉重來;系統上線之後,每一次改動都要跟「已經存在的資料」和「正在跑的舊版程式」共處。這份文件記的就是後者的做法。

核心原則

- **由內而外**:Domain → Configuration → Migration → API → 前端。設定檔只是在描述 Domain 已經有的東西,不能反過來。
- **加法變更(additive)**:新增可為 null 的欄位是最安全的一種 schema 變更,不需要重建資料表,舊資料也不用回填。
- **手動改 > 重新產生**:重新產生整份 solution 會蓋掉你手寫的商業邏輯與註解。這個變更只碰四到六個檔案,手動完全可控。

操作步驟

1 Domain 層 Reservation.Domain

```
public sealed class Reservation
{
    public string? Note { get; private set; }      // nullable、private set

    public void UpdateNote(string? note)
    {
        if (note is { Length: > 200 })
            throw new ArgumentException("備註長度不可超過 200 字。");

        Note = note;
    }
}
```

不要加進建構子。多一個必填參數,等於打斷所有既有的呼叫端與測試。用「可為 null 的屬性 + 一個明確的行為方法」才是加法變更。方法名也要符合 ubiquitous language —— 是 `UpdateNote`,不是 `SetNote`。

2 先補測試 Reservation.Domain.Tests

- 新建的 reservation, Note 應為 null
- `UpdateNote("靠窗")` 之後讀得回來
- 超過 200 字要丟例外
- `UpdateNote(null)` 是合法的「清除備註」

3 EF 設定 Reservation.Infrastructure

```
// ReservationConfiguration.cs
builder.Property(r => r.Note)
    .HasMaxLength(200);      // 對應 nvarchar(200),預設 nullable
```

Domain 的 200 是**第一道防線**,資料庫的 `nvarchar(200)` 是**第二道** —— 跟複合主鍵那裡的思路一致。兩個數字必須同步,改一個就要改另一個。

4 產生 Migration

```
dotnet ef migrations add AddReservationNote \
--project Reservation.Infrastructure \
--startup-project Reservation.Api
```

產生完一定要打開來看。這是最多人跳過、也最容易出事的一步。

應該看到:

```
migrationBuilder.AddColumn<string>(
    name: "Note",
    table: "Reservations",
    type: "nvarchar(200)",
    maxLength: 200,
    nullable: true);
```

如果看到的是 `DropTable` / `CreateTable` —— **停手**。表示 model 有其他地方漂移了,EF 決定重建資料表,你的正式資料會消失。

在 SQL Server 上,新增一個 nullable 欄位是 **metadata-only** 操作:瞬間完成,不重寫任何一列資料,即使資料表已經有幾百萬筆也一樣。這正是「加法變更」值錢的地方。

5 套用

```
# 開發機
dotnet ef database update --project Reservation.Infrastructure --startup-project Reservation.Api

# 正式機 — 不要直接 update,產生 SQL 讓人審過再跑
dotnet ef migrations script --idempotent --output add-note.sql
```

`--idempotent` 產生的腳本會自己判斷有沒有跑過,重複執行也安全。

6 往外層擴散:容易漏掉的清單

位置	要不要改	說明
Repository	通常不用	EF 會自動帶上新欄位,除非你有手寫 <code>Select</code> 投影
手寫的 projection	要	<code>.Select(r => new ...)</code> 不會自動長出新欄位,靜悄悄漏掉
Request DTO	要	加上 <code>string? Note</code> ,並補上長度驗證
Response DTO	要	同上
Mapping	要	手寫 mapping 漏一行不會編譯錯誤,只會永遠是 <code>null</code>
API endpoint	視情況	建議獨立一支 <code>PATCH /reservations/{id}/note</code> ,對應 domain 的行為方法
Next.js 前端	要	表單欄位、型別定義、顯示畫面; <code>maxLength</code> 要跟後端一致
整合測試	要	存檔 → 換一個新的 <code>DbContext</code> → 讀回來,確認真的有進資料庫

Mapping 漏一行是本次最可能的失誤。它不會編譯錯誤、不會拋例外、不會有任何警告 —— 使用者填了備註、按了送出、畫面顯示成功,而資料庫裡是 `null`。跟先前 `_tables.ToList()` 那個 getter 是同一類的坑:**錯了但不報錯**。

為什麼一定要 nullable

資料庫裡已經有舊的訂位資料。如果設成 `NOT NULL`,你就必須替那些「在這個概念存在之前就建立」的訂位編一個值出來:

- `""` 是說謊 —— 它的意思是「我們問過了,客人沒有特殊需求」
- `null` 才誠實 —— 「我們根本沒問過」

另一個好處是**部署順序不重要**。舊版程式忽略這個欄位照跑,新版程式處理 `null` 也沒事,先上資料庫或先上程式都可以。這叫 `expand / contract pattern` —— 「兩種順序都安全」的變更,跟「必須照順序來」的變更是完全不同等級的風險。

設計提醒:素 / 葷 / 穆斯林 不該放在 Note

這是一個**封閉集合**,而且對廚房有實際的作業意義,不是備忘錄。放進自由文字,一個月內那個欄位裡會出現:

素 / 素食 / 全素 / 蛋奶素 / vegetarian / no meat / 不吃豬 / 不吃牛

結果是:沒有人能查詢、廚房算不出份數、也答不出「明天有幾份清真餐」。建議拆成兩個欄位:

```
public DietaryRequirement Dietary { get; private set; } // enum:可查詢、可統計
public string? Note { get; private set; } // 真正的一次性備註
```

Migration 成本一模一樣 —— 兩個 `AddColumn` 而不是一個。但這個決定**只有一次便宜的機會**:等自由文字進了正式資料庫,再要拆就得寫資料清洗的 migration,對著真實資料一筆一筆處理,那才是貴的。

一頁檢查清單

檔案 / 位置	動作
☐ Reservation.cs	加 <code>public string? Note { get; private set; }</code>
☐ Reservation.cs	加 <code>UpdateNote(string?)</code> ,含長度檢查
☐ Domain.Tests	四個測試:預設 null、更新、超長丟例外、清除
☐ ReservationConfiguration.cs	<code>Property(r => r.Note).HasMaxLength(200)</code>
☐ CLI	<code>dotnet ef migrations add AddReservationNote</code>
☐ Migrations/*.cs	打開來讀 :必須是 AddColumn,不能有 DropTable
☐ CLI	開發機 <code>database update</code> ;正式機產生 idempotent script
☐ 手寫 projection	搜尋所有 <code>.Select(</code> ,確認有沒有漏帶 Note
☐ Request / Response DTO	加欄位 + 驗證(<code>MaxLength(200)</code>)
☐ Mapping	雙向都要,漏了不會報錯
☐ API	新增或擴充端點
☐ Next.js	型別、表單、顯示、maxLength
☐ 整合測試	換新的 DbContext 讀回來才算數

踩雷紀錄(給「卡關日誌」)

現象	原因
Migration 產生了 DropTable / CreateTable	model 有其他漂移;或 <code>ApplyConfigurationsFromAssembly</code> 沒抓到你的 configuration(類別不是 public、沒有無參數建構子),導致 EF 退回慣例對應
備註存不進去,但沒有任何錯誤	mapping 漏了一行,或手寫的 <code>Select</code> 沒帶到
測試通過,實際跑卻是 null	測試用了同一個 DbContext —— 變更追蹤把記憶體裡的物件還給你,根本沒碰資料庫
資料庫允許超過 200 字	只改了 Domain 沒改 Configuration,或反過來。兩道防線要同步
正式機 update 失敗	Ubuntu VM 連 Windows 主機的 SQL Server:確認 Host-Only IP、TCP/IP 已啟用、防火牆放行 1433